BUILD PROMPT: Heterogeneity Under Observability (HUO) Metric Toolkit

What this document is. A build prompt for a self-evaluation toolkit that tests the Heterogeneity Under Observability (HUO) framework's arithmetic and metric procedure. Give it to any capable AI, and it will produce three connected deliverables. The tools let a program manager check the framework's internal consistency and metric repeatability without trusting the author's claims.

What you get back. Deliverable 1 is a Python package that runs the paper's toy computations and nine consistency experiments, including a dry run of the evaluation protocol against a mock adversary agent. Deliverable 2 is an interactive analyst worksheet, delivered as both a single-file HTML tool and a Python command-line interface, that applies the metric procedure to a real or hypothetical variant family and produces a reproducible report. Deliverable 3 is a six-to-ten-page pilot design, ready to adapt into a funded proposal, for a small program that would run the evaluation protocol against a real adversary agent.

What you do not get back. Any measurement of real adversary cost. Any validation of the framework against real systems. Any claim that heterogeneity works in practice. Those require the funded pilot described in Deliverable 3, not the toolkit itself. The toolkit tests whether the formulas are internally consistent and whether the metric is repeatable across analysts. Those are necessary but not sufficient conditions for the framework to be useful.

How to use it. Give this document to a capable AI, along with the paper if the tool accepts attachments (HETEROGENEITY_UNDER_OBSERVABILITY_FULL.pdf or the Markdown source). If the AI cannot accept the attachment, Appendix A below is a complete enough specification to work from. Where the paper and Appendix A disagree, Appendix A governs. Ask for the three deliverables in the order given. Deliverable 2 is the one most useful to a program manager evaluating the framework; if session budget is limited, prioritize it.

Expected runtime. Two to six hours of AI execution time for a capable model, plus thirty to sixty minutes of human review to verify the test-vector table and spot-check the analyst worksheet. These are estimates, not measurements.

The prompt follows.

1. Your role

You are a senior research engineer and technical writer. You will build three connected deliverables that support a response to DARPA RFI DARPA-SN-27-17 (FLASH), titled "Heterogeneity Under Observability: An Acquisition-Cost Framework for Assessing Diversity as a Cyber-Physical Defense." The author is AionSystem (Technical POC Sheldon K. Salmon). The RFI response is due November 30, 2026.

The paper argues that whether design diversity (heterogeneity) raises an adversary's cost depends on how the adversary acquires knowledge. It proposes a metric, H_d,variant, that predicts the adversary's burden, and an evaluation protocol that tests the prediction. The protocol has not been run.

The three deliverables:

Synthetic-family simulation. A runnable Python package that implements the metric procedure and the adversary burden model, and tests them on generated variant families with injected, documented correlations.

Analyst worksheet and calculator. A tool that walks a human analyst through the metric procedure on a real or hypothetical variant family and produces a declared, reproducible report.

Pilot design. A written design for a small funded pilot that would run the evaluation protocol for real.

Build them in the order given. Each later deliverable reuses the earlier one's logic.

When you finish, you will return the report described in section 6. If you cannot complete all three deliverables, return the report with the completed sections and the reason for the incomplete ones. Do not skip the report.

2. Ground rules (read twice)

Honesty about what this is.

The simulation tests whether the method and the formulas behave as the paper describes. It does not show that the metric predicts real adversary cost. That claim is what the real protocol tests.

Every output (code docstrings, README, plots, reports, console banners) must label simulated adversary times as "mock" or "illustrative." Never write "validated," "proven," "demonstrated that heterogeneity works," or similar.

Do not invent citations, data, vendor names, CVEs, or results. The only literature the paper cites is: Knight and Leveson 1986 (multiversion programming independence experiment), Littlewood and Miller 1989 (coincident failure modeling), and Larsen et al. 2014 (SoK: Automated Software Diversity). Do not add others unless you can state exactly what they are and flag them "verify before citing."

If you are unsure of a fact, say so in the report. Do not fill gaps with plausible text.

Engineering standards.

Python 3.10+, standard library plus numpy, scipy, pandas, matplotlib. No network access at runtime. Deterministic given a seed. Every random call goes through one seeded generator.

Every formula in Appendix A must have at least one unit test using the test vectors in Appendix B.

Code must run end to end with a single command per deliverable and print a short pass/fail summary.

Write clear docstrings that cite the paper section (for example "paper section 7.2, Step 3").

Writing style for every document you produce.

Plain, direct prose. Avoid em dashes; use commas, colons, or parentheses instead.

Define notation on first use. Write subscripts in plain text (H_d,variant, c_capture) in code and Markdown.

No marketing language. State limits next to claims.

Safety.

Nothing you build may contain working exploit code, real vulnerability details, or instructions for attacking real systems. The "adversary" in the simulation is a statistical model of time and cost. In the pilot design, any autonomous exploit-development agent must run only against purpose-built or explicitly consented targets in an isolated sandbox. State this in the pilot document.

Process.

Work in the order: simulation, worksheet, pilot design.

Ask at most one clarifying question, and only if you cannot proceed. Otherwise make a reasonable assumption and list it in ASSUMPTIONS.md.

After each deliverable, run its tests, and report exactly what passed and what you did not verify.

When you cannot complete a section. Produce as much as you can, label every incomplete section with "INCOMPLETE" in the heading, explain in the final report what prevented completion, and proceed to the next deliverable. Do not skip a deliverable silently. Do not produce placeholder text that looks finished.

Mock-data labeling. Every figure, table, and summary line produced from experiment E9 (the mock protocol dry run) must begin with the words MOCK DATA in capitals. No number from E9 may appear in any output without that prefix. No comparison between E9 results and real-world behavior may appear anywhere in the output. If a table mixes E9 with non-E9 rows, split it into two tables.

3. Deliverable 1: Synthetic-family simulation

3.1 Purpose

Check the internal consistency of the paper's formulas and metric procedure, and give the author a reproducible demonstration (labeled as illustrative) of the following: the formula arithmetic; the H_d,node versus H_d,variant divergence; hidden centralization collapsing the burden toward the homogeneous case; cluster and topology effects; analyst-dependence; and the O(N^2) defender cost.

3.2 Package layout

```
huo_sim/
  README.md
  ASSUMPTIONS.md
  pyproject.toml            (or requirements.txt)
  huo/
    model.py        data classes: Node, Variant, Family, TransferModel
    generator.py    synthetic family generator with injected correlations
    metric.py       independence test, node ratio, minimum-path reduction, variant fraction
    burden.py       adversary cycle counts and costs for A1, A2-hybrid, A2-pure
    analyst.py      analyst noise model for J-1, J-2, J-3 and theta
    stats.py        protocol statistics and the outcome decision table
    mock_agent.py   MOCK exploit-development time generator (clearly labeled)
    experiments.py  runs experiments E1 to E9 below, writes CSV and PNG
  tests/
    test_vectors.py all tests from Appendix B
  outputs/          generated CSVs, plots, and a summary report (Markdown)
```

3.3 Data model

Node: id, technique_class (the latent "exploit technique" it belongs to), base_effort_hours, consequence weight, component_class (for J-1 functional-class granularity), artifact_id (for specific-artifact granularity).

Variant: id, set of critical nodes at declared depth d.

Family: list of variants, generation specification id, optional "generator_bias" parameter.

TransferModel: for any ordered pair (node X in variant v, node Y in variant w), the fractional reduction in exploit effort against Y after studying X in v. Default rule: if X and Y share a technique_class, discount = d_transfer (default 0.6, configurable, with optional noise); otherwise 0. Generic background knowledge (J-2) is modeled as a constant discount applied equally to all nodes and is excluded from transfer.

3.4 Generator knobs

The generator must let the user set, and must record in the output, ground truth for each family:

Number of variants N (2 to 200) and nodes per variant.

Number and size of shared nodes (present in all variants) and partially shared nodes.

Number of technique classes, and the probability that variant-specific nodes share a technique class with another variant's nodes (this creates pattern-level transfer).

Hidden centralization injection: for each of the paper's eleven mechanisms (a) to (k) (Appendix A.6), a switch to inject it. Mechanisms (a), (b), (c), (f), (g), (h), (i), (k) are modeled as shared or transferable nodes. Mechanism (d), shared generator bias, is modeled by the generator_bias parameter that makes designs predictable from a small sample, and must trigger the family-level check. Mechanisms (e) and (j) are specification-level: model them as a flag spec_level_exploit=True that, per limitation L4, forces the true burden to the homogeneous case regardless of H_d,variant.

Cluster structure: ability to place non-independent variants into k_clusters transfer-equivalence classes.

Per-node effort distribution (lognormal by default), with a setting so that the shared node is, or is not, the cheapest path.

Ground-truth labels: true independent-variant set, true k_clusters, true cheapest paths.

3.5 Metric implementation (must follow Appendix A.3 exactly)

Implement the independence test with threshold theta, the node ratio H_d,node, the minimum-path reduction with the largest-single-discount rule and the 20% co-cheapest band, the variant fraction H_d,variant, the family-level generator check (flag plus override to 0), and the J-1 granularity switch. Report the conventions used and the known direction of error alongside every result.

3.6 Burden model

Implement, for N variants, H_d,variant, and costs c_capture, c_exploit, c_public:

The continuous formula 1 + H*(N-1), the exact discrete cycle count (including k_clusters and the endpoint cases), and its ceiling.

Cost for A1 (c_public per cycle), A2-hybrid (topology-adjusted: captures = max(1, ceil(cycles / k_cap)), total = captures * c_capture + cycles * c_exploit), and A2-pure (flat per-engagement cost).

Success fraction f (effective N = f*N) and single-variant deployment collapse (N = 1).

A discrete-event adversary simulator that actually plays out the study-and-exploit process on a generated family (adversary picks a variant, pays costs, applies discounts to others) so that the closed-form count can be compared to a simulated count. This is the main internal consistency check.

3.7 Experiments to implement (write CSV plus a PNG for each)

E1 Formula vs exact count. For N in 2..50 and every k = H*N, compare the continuous formula, its ceiling, and the exact discrete count. Expected: ceiling equals exact for all 0 <= k <= N; continuous formula equals exact at endpoints and is below k+1 by H in between.

E2 Node ratio vs variant fraction. Reproduce the paper's worked example (Appendix B, vector V2), then sweep over how cheap the shared node is, and plot H_d,node vs H_d,variant and the resulting error in predicted cycles.

E3 Simulator vs formula. For many random families with no hidden centralization, compare simulated cycle counts with the closed form. Report agreement and every disagreement with a cause.

E4 Hidden centralization. Inject each mechanism (a) to (k) one at a time and show how the node-level test, the family-level check, or the L4 scope limit catches it or fails to. Report which mechanisms the metric misses by design.

E5 Cluster and topology. Sweep k_clusters and k_cap. Show the cycle-count error from the single-cluster assumption and the capture-count effect of topology.

E6 Sensitivity. Sweep theta (for example 5% to 50%), the co-cheapest band, and J-1 granularity. Plot how H_d,variant moves. This quantifies the paper's declared analyst-dependence.

E7 Analyst noise. Simulate two analysts who apply the procedure to the same family with noisy effort estimates and noisy transfer judgments. Report the distribution of the difference in H_d,variant. This previews the pilot's analyst-repeatability measurement.

E8 Defender cost scaling. Time the pairwise independence computation as N grows and fit the exponent, to check the O(N^2) worst-case claim. Report the fitted exponent, not just the plot.

E9 Protocol dry run (MOCK). Use mock_agent.py (a lognormal exploit-time model whose parameters are set so that time scales with the true number of independent cycles, with explicit noise) to run the full protocol pipeline: calibration with at least 5 runs and a T_cal budget, baseline T_f1, a sweep of H_d,variant, at least 5 runs per (H_d,variant, class) cell, the 95% confidence interval of the observed ratio, the check against the predicted ratio ceil(1 + H*(N-1)) within a declared tolerance (default factor 2), and the outcome decision table in Appendix A.5. Include one run where the mock agent is deliberately made to violate the hypothesis (for example flat time under A2-hybrid) and confirm the pipeline reports "Falsified." Every figure, table, and summary line from E9 must begin with "MOCK DATA" in capitals. No number from E9 may appear in any output without that prefix.

3.8 Acceptance criteria for Deliverable 1

All Appendix B test vectors pass.

E1 to E9 each produce CSV, PNG, and a paragraph in outputs/summary.md stating what the experiment shows and what it does not.

summary.md ends with a "What this does not show" section stating that no real adversary cost was measured.

A README with install steps, a one-command run, and the expected runtime.

4. Deliverable 2: Analyst worksheet and calculator

Note: this is the deliverable most useful to a program manager evaluating the framework. If you have limited session budget and cannot complete all three deliverables, complete this one and Deliverable 3, and mark Deliverable 1 INCOMPLETE with an explanation.

4.1 Purpose

Let a human analyst apply the metric procedure to a real or hypothetical family, record every judgment, and get a reproducible report. This also supports the pilot's analyst-repeatability test.

4.2 Form

Build both:

A single-file HTML tool (HTML, CSS, JS in one file, works offline, no external requests, no browser storage; provide Export JSON and Import JSON buttons to save and restore a session).

A Python CLI (huo_calc) that reads the same JSON and produces the same results, built on the Deliverable 1 metric.py and burden.py modules. A test must confirm the HTML tool and the CLI agree on all Appendix B vectors.

4.3 Workflow the tool must support

Declarations: analyst name or ID, date, adversary capability class (free text plus a short list), analysis depth d, criticality threshold, theta (default 20%), co-cheapest band (default 20%), node granularity choice (J-1: specific artifact by default, functional class as an option), the J-2 rule text (generic transfer does not count), and the J-3 unit (expected adversary hours).

Family entry: variants and their critical nodes (name, artifact ID, component class, consequence weight). Support paste-in CSV and manual entry. Provide an example loader for the Appendix B vectors.

Effort entry: expected adversary hours per (node, variant). Allow a low and a high estimate to carry the analyst's range.

Transfer entry: for node pairs across variants, the estimated fractional effort reduction from studying one to attack the other. Provide a matrix view and a "same technique class" shortcut. Flag blank cells.

Computation, shown step by step with intermediate tables: independence test per (node, variant) pair, common versus variant-specific classification, C_c and S_c, H_d,node, per-variant cheapest path with co-cheapest handling and the largest-single-discount rule, per-variant independent or non-independent classification, H_d,variant, transfer clusters (connected components among non-independent variants) giving k_clusters, and the exact discrete cycle count.

Burden output for A1, A2-hybrid, A2-pure: continuous formula, discrete count, topology-adjusted captures given k_cap, total cost given user-supplied c_public, c_capture, c_exploit (each as a low-high range), and f.

Range reporting: results at the low and high ends of the analyst's range, with the formula value at both ends, as the paper requires.

Warnings panel (these are required):

A shared node is the cheapest path for some variant while H_d,node is high (the node-ratio trap).

The family-level generator check has not been answered. It is a human judgment; the tool must force an explicit yes or no, and "yes" sets H_d,variant to 0.

Specification-level and in-envelope exploitation (mechanisms e and j) are outside the metric (limitation L4); the tool must display this notice on every report.

More than one transfer cluster, so the single-cluster formula may understate the cycle count.

Any cell left blank or any J choice left at default without confirmation.

Depth truncation: nodes below depth d are counted as common, which lowers H_d,variant.

Direction-of-error statement auto-generated from the choices made (conventions that lower H_d,variant versus conventions that can raise it, per Appendix A.3).

Report export: a Markdown report (and print-friendly HTML) containing declarations, inputs, every intermediate table, results with ranges, warnings, conventions, and a closing line "Output of the analyst worksheet; not evidence of adversary cost."

Two-analyst comparison mode: load two exported sessions on the same family and report the difference in H_d,variant, which variants were classified differently, and which judgment (J-1, J-2, J-3, theta) drove each difference. This is the pilot's "analyst-dependence residual."

4.4 Acceptance criteria for Deliverable 2

The HTML tool and CLI reproduce all Appendix B vectors identically.

A worked example for each vector is included as a loadable sample.

The tool never silently defaults a judgment point; every J choice is shown and confirmed in the report.

A short user guide (one to two pages) explains each step in plain language for a non-expert analyst.

5. Deliverable 3: Pilot design document

5.1 Purpose

A written design, ready to adapt into a funded proposal, for a small pilot that runs the paper's evaluation protocol on one variant family. It must be honest that the pilot may falsify the metric.

5.2 Format

One Markdown document, about 6 to 10 pages, with numbered sections and tables, plus a one-page executive summary at the front that a program manager can read in two minutes. Use plain language and define every symbol.

5.3 Required contents

Objective and hypotheses. State the central hypothesis (H_d,variant predicts adversary burden under A1 and A2-hybrid, and heterogeneity has no effect under A2-pure for static or slow-dynamic heterogeneity) and the three falsifiers from Appendix A.5.

Task 0, domain gate. Verify the chosen domain has documented cross-variant failure history, as the paper's section 7.5 requires. State the fallback options: published multiversion experiments (Knight and Leveson), and synthetic families with injected, documented correlations (reuse Deliverable 1's generator). Define the go or no-go rule.

Candidate variant families. List 3 to 5 candidate family types, with for each: why it might satisfy the gate, what data would exist, risks, and what you could not verify. Do not assert facts about specific products or projects that you cannot confirm; mark any such claim "to verify." Describe candidate types in general terms (for example, independently written implementations of one specification, open-source forks of one codebase, diversified builds of one codebase) and say what each can and cannot test.

Metric measurement task. How H_d,variant will be computed with the Deliverable 2 worksheet, including the analyst range, theta, depth, J choices, and recording k_clusters and deployment topology (k_cap).

Analyst repeatability study. Two independent analysts apply the procedure to the same variant pair or small set. Define what is compared, the measure of divergence, and how the result is reported (the "analyst-dependence residual").

Dominant-class determination. How the paper's four inputs (capture exposure, extractability, remote paths, live observability) would be assessed for the chosen setting, and what evidence supports the declared class.

Adversary agent specification. The autonomous exploit-development agent's declared capability class and required abilities (adaptive search, hypothesis revision, independent operation). Calibration: at least 5 runs against a named known-vulnerable single-variant baseline, each reaching a working exploit within budget T_cal, or the run is void. State that T is agent time, mapped to adversary cost only under the declared capability class. Include a sandboxing and ethics section: isolated environment, purpose-built or consented targets only, no real-world systems, logging, and an authorization step.

Experimental matrix. A table of cells: H_d,variant levels (from about 0 to high) by adversary class (A1, A2-hybrid, A2-pure), at least 5 runs per cell (fewer reported as preliminary). For each class, the declared channel: A1 sees specification, public documentation, one reference variant, with no channel into a deployed instance; A2-hybrid has a named capture duration, firmware-extraction path or intrusion path,

and a declared k_cap; A2-pure has named live probing and side-channel access with no persistent access. Include f (fraction of fleet) and note effective N = f*N.

Measurements and statistics. T_f per cell, whether mean, median, or geometric mean is quoted (state a default and justify it, since search times are right-skewed), the predicted ratio ceil(1 + H*(N-1)), the 95% confidence interval of the observed ratio, the tolerance (default factor 2), and the pre-registered outcome table (Appendix A.5). Include the defender-cost measurements: per-variant verification effort and the effort of computing H_d,variant, recorded in the same experiment.

Capture cost estimation. T measures exploit-development time after the channel is granted. Explain how c_capture is estimated separately so that T alone is not mistaken for c_capture + c_exploit.

Threats to validity. At least: FMEA-to-exploit proxy risk (the bridge is asserted, not proven), analyst dependence, agent-versus-real-adversary gap, small N, synthetic-versus-real family gap, hidden centralizations the metric cannot price (mechanisms e and j), and deployment collapse (N = 1 in the field).

Deliverables and decision rule. The pilot ends with a validated or a falsified metric. Define what each outcome means for next steps, and what outcomes change the practical claim (A1 dominance everywhere, or A2-pure dominance in the RFI's application examples).

Plan. A phase table (Task 0 gate, family selection, metric measurement, analyst study, agent calibration, experiment runs, analysis and report) with placeholders for duration, staffing, and cost. Do not invent dollar figures; use clearly marked placeholders.

Open items. A list of what the author must decide or supply.

5.4 Acceptance criteria for Deliverable 3

Every protocol parameter that appears in Appendix A.4 and A.5 is present and consistent with it.

The document states plainly that the pilot may falsify the metric.

All unverifiable factual claims about real software, vendors, or datasets are marked "to verify."

The one-page executive summary stands alone.

6. Final report you must return

When all three are done, return a single wrap-up message containing:

A file tree of everything you produced.

How to run each deliverable (one command each).

A table of every Appendix B test vector with pass or fail.

A list of every assumption you made (also in ASSUMPTIONS.md).

A list of anything in the paper or this prompt that looked inconsistent, ambiguous, or wrong, with the section reference and your proposed fix. Do not silently correct the paper's logic. Report it.

A plain statement of what you did not verify.

Appendix A: Specification (authoritative)

A.1 Setting

A design family F = {v_1, ..., v_N} is a set of functionally equivalent implementations generated from one specification, deployed so each member is a realization. Heterogeneity is a property of the family. The anchor application class in the paper is unmanned surface and undersea vehicles (USV/UUV) doing autonomous maritime patrol, but the metric is domain-neutral.

Three conditions for heterogeneity to pay: (a) genuine degrees of freedom that preserve required behavior; (b) the adversary must acquire knowledge of each variant, which excludes full state-observability through real-time probing; (c) per-variant verification cost does not exceed the value of the diversity. Condition (b) decides whether heterogeneity pays.

A.2 Adversary classes (by acquisition mode)

A1, class-attacking. Studies the design family from public or purchased information; never accesses the deployment. Unit cost c_public per cycle.

A2-hybrid, capture-then-exploit. Captures one deployed instance (physical capture or cyber intrusion), studies it with time, and deploys exploits across the fleet. Unit cost c_capture + c_exploit per cycle.

A2-pure, real-time. Probes the deployed instance transiently, with no capture and no study. Burden is 1 x per-engagement cost.

Burden by class, with H = H_d,variant:

Homogeneous family (H = 0): A1 = 1 x c_public; A2-hybrid = c_capture + c_exploit (fleet compromised); A2-pure = 1 x per-engagement cost.

Heterogeneous family (H > 0): A1 = [1 + H(N-1)] x c_public; A2-hybrid = [1 + H(N-1)] x (c_capture + c_exploit); A2-pure = 1 x per-engagement cost.

Effect: A1 and A2-hybrid share the same structural multiplier at different unit costs. For static or slow-dynamic heterogeneity, A2-pure is unaffected. Whether the absolute value is larger under A2-hybrid depends on whether c_capture + c_exploit exceeds c_public. The classes are a taxonomy, not an exclusive typing (an adversary can pair A1 preparation with A2-pure engagement); the burden calculation uses the class that dominates the adversary's work.

When the A2-hybrid formula holds (assumptions):

No missed transfer: the independence test found every transfer between variants.

H_d,variant is the post-test, post-reduction value (not the node ratio).

Cluster structure: the non-independent (1 - H) variants form one transfer cluster. With k_clusters transfer-equivalence classes, the exact cycle count is H*N + k_clusters (k_clusters = 0 when no variant is non-independent).

Topology-adjusted capture: captures = max(1, ceil(cycles / k_cap)); total cost = captures x c_capture + cycles x c_exploit, where k_cap is the average number of variants reachable per capture.

If a shared vulnerability exists (any of mechanisms a to k), the real cost is closer to the homogeneous case.

Exact discrete count for single transfer cluster: 1 if H = 0; N if H = 1; HN + 1 for 0 < H < 1. The continuous formula 1 + H(N-1) equals the exact count at both endpoints, is below H*N + 1 by H in between, and its ceiling equals the exact count for every H in [0, 1]. The paper's checks confirm arithmetic only.

Success fraction. For f < 1 the effective N is f*N. A family deployed as a single variant has N = 1 for the fielded subset regardless of design-time H.

Defender burden. N x per-variant verification (no variant inherits another's assurance), plus the cost of computing H_d,variant, which is pairwise and O(N^2) in the worst case. Both defender costs and the adversary's cost (under A1 and A2-hybrid) are incurred once per variant set, because an exploit against an unchanged variant persists. Repeated attacks add to the adversary's cost only where its knowledge is invalidated (a replaced or patched variant, a closed capture channel), and the defender then pays to verify the replacement. The comparison is per variant set, not per attack.

Dynamic heterogeneity. Observability becomes predictability. If the variant sequence follows observable rules, the adversary specializes against it (the A2-hybrid case at fleet scale). If randomized over a space the adversary cannot enumerate, the adversary must specialize against the distribution (A1 with added cost). Under A2-hybrid, unpredictability requires that the entropy source and selection policy cannot be recovered from the captured instance. The dynamics must be unpredictable to the adversary and verifiable by the defender. Static analysis here is scoped to static and slow-dynamic heterogeneity; faster-than-engagement dynamics are a different mechanism not analyzed.

A.3 The metric procedure (paper section 7.2)

Subject. Family F. Critical nodes N_c(v, d): components of variant v whose failure produces system-level failure within the declared model scope, weighted by consequence, at declared depth d. The criticality threshold is declared. Nodes below depth d are unresolved and counted as common.

Step 1, independence test. A node X in variant v that appears in no other variant is variant-specific if no technique developed against X gives material advantage against any node in any other variant, and common otherwise. Material advantage means an adversary-cost reduction of at least theta (default 20%); a transfer below theta counts as none. The test is binary and operates on (node, variant) pairs for partially shared nodes. A partially shared node is counted common if exploitation of it in one variant gives material advantage against a variant lacking it (pattern transfer). C_c = nodes in every variant plus variant-specific nodes whose exploitation transfers above theta. S_c = variant-specific nodes whose exploitation does not transfer above theta to any variant not already containing it.

Step 2, node ratio (diagnostic only). H_d,node(F) = |S_c| / |union over v of N_c(v, d)|.

Step 3, minimum-path reduction. For each variant, find the cheapest critical path, measured as expected adversary effort in hours for a declared capability class. Effort = per-node exploit development effort minus the transfer discount from studying any single other variant (the largest discount applies; discounts from studying several variants are not cumulated). Paths within 20% of each other are co-cheapest; the variant is non-independent if either co-cheapest path is common, and independent only if both are variant-specific.

Step 4, variant fraction (what the formula uses). H_d,variant(F) = |{v in F : cheapest critical path of v is variant-specific}| / |F|.

Family-level generator check. If the family's designs come from a process whose biases are inferable from a small sample of designs, the family is non-independent and H_d,variant is reported as 0. This is a required human judgment, not a computable quantity.

Analyst-judgment points (declare all three in every report).

J-1, node granularity. Default: a specific artifact (part number plus firmware version). Alternative: functional class (for example "radar" as one node), which classifies more nodes as common and lowers H_d,variant.

J-2, generic versus specific transfer. Generic transfer (a technique any adversary could develop without capturing a variant) is background knowledge and does not count. Only a technique developed against a specific variant and shown to give material advantage against another counts. This also closes mechanism (d) at the node level: low-entropy generator choices inferable from the public specification are background; those inferable only from a captured design are transfer.

J-3, cheapest path in what unit. Expected adversary effort in hours for a declared capability class; 20% co-cheapest band (a declared design choice).

Direction of error. H_d,variant approximates H_variant (the fraction under the true critical failure set) with error in both directions. Conventions that lower it: nodes below depth d counted as common; the co-cheapest rule. Conventions that can raise it: the J-1 specific-artifact default; the binary threshold theta with sub-theta discounts not cumulated; a false no-transfer call in Step 1. Reports list the conventions used. Specification-level exploitation and in-envelope-behavior exploitation (mechanisms e and j) are outside H_d,variant (limitation L4): under them the burden is the homogeneous case regardless of H.

Bridge assumption. FMEA failure nodes are a proxy for exploitable surfaces. The proxy is asserted, not proven. The formula has no usable input if this fails; the empirical protocol is the test.

Independent check of H_d,variant (paper section 7.5). Compute H_d,variant on families with documented cross-variant failure history. Where failures recurred across variants, H_d,variant should be low; where variant-specific, high. This tests only the recurrence correspondence. If the domain lacks such history, use published multiversion experiments (Knight and Leveson) or synthetic families with injected, documented correlations.

A.4 Evaluation protocol (paper section 9)

Six criteria. A heterogeneity claim is assessable only if it states: (1) the variant family and critical node set with J-1 to J-3 applied and the independence test and minimum-path reduction run; (2) the measured H_d,variant, depth, and analyst's range; (3) the adversary class with the observation and capture model specified; (4) time-to-compromise T_f under a calibrated autonomous exploit-development agent, with f declared (f = 1 for full fleet; for f < 1 effective N = f*N); (5) the correspondence between H_d,variant and T_f as a test with interpretation fixed in advance; (6) the defender's cost (per-variant verification effort and effort of computing H_d,variant) recorded in the same experiment.

Protocol. Calibration: run the agent (capable of adaptive search, hypothesis revision, independent operation) against a named known-vulnerable single-variant baseline; at least 5 runs must each reach a working exploit within budget T_cal, or the run is void. Baseline: a single-variant family, T_f1. Heterogeneous families: sweep H_d,variant from about 0 to high and record T_f for f*N variants. Three adversary conditions (A1, A2-hybrid, A2-pure), channel declared per experiment. Capture cost: T measures exploit-development time after the declared channel is granted; c_capture is set by the channel declaration and estimated separately, so T alone does not measure c_capture + c_exploit. Defender cost recorded for each family.

A1	A2-hybrid	A2-pure	
Sees	Specification, public documentation, one reference variant	Deployed instance through a declared capture channel, with time to study	Deployed instance through live observation, without study
Channel	None into the deployed instance	Named: capture duration, firmware extraction path, intrusion path	Named: live probing, side-channel access, no persistent access
Topology	n/a	k_cap declared; sets the number of captures	n/a

An experiment that does not state its channel does not produce a class-attributable result.

Statistical specification. At least 5 runs per (H_d,variant, class) cell; fewer is preliminary. The prediction is multiplicative: for f = 1, predicted ratio T_f(H)/T_f(0) = ceil(1 + H*(N-1)). Support requires the 95% confidence interval of the observed ratio to exclude 1 and lie within a declared tolerance of the prediction (default factor 2). Search times are right-skewed, so state whether mean, median, or geometric mean is quoted. Name the baseline. Give the analyst's H_d,variant range with the formula value at both ends.

A.5 Outcome table (fixed before any run)

Observation	Interpretation
T rises with H under A1 and A2-hybrid; flat under A2-pure	Supported for both classes; the multiplier is shared
T rises under A2-hybrid; flat under A2-pure (A1 not run)	Supported for A2-hybrid
T flat under A1; rises under A2-hybrid	Structural-symmetry claim falsified
T rises under A1; flat under A2-hybrid	Falsified for A2-hybrid
T flat under all three	Falsified; H_d,variant is not predictive
T rises under all three	Falsified; heterogeneity imposes cost under A2-pure, which the paper did not predict
T flat under A2-hybrid; rises under A2-pure	Falsified in the opposite direction
T_cal not met	Run void

Falsifiers of the argument: flat time-to-compromise under A2-hybrid; H_d,variant uncorrelated with documented failure history; T rising under A2-hybrid but flat under A1.

Program-feasibility conditions: no architecturally independent assurance path buildable; H_d,variant not computable at a useful depth; specification-formalization path not implementable.

Outcomes that change the practical claim: A1 dominance in every RFI example (structural multiplier at a lower absolute per-variant cost); A2-pure dominance in the RFI's examples (removes the benefit of static heterogeneity).

A.6 The eleven hidden-centralization mechanisms

The adversary's knowledge transfers across variants that share: (a) a software stack; (b) a supply chain or single-vendor components; (c) a physical weakness in a material or process; (d) a generator whose low-entropy choices point at the family's failure surface; (e) a specification or protocol; (f) trust anchors, keys, or update channels; (g) a toolchain; (h) an environmental dependency such as GNSS; (i) a deployed perception or classification model, its weights, or its training data; (j) in-envelope behavior, because the invariant set requires equivalent behavior inside the envelope, so an exploit acting through specified behavior applies to every variant; (k) sensing phenomenology, such as radar or acoustic spoofing and jamming. The node-level test finds (a) to (c), (f) to (i), and (k). Mechanism (d) needs the family-level check. Mechanisms (e) and (j) are specification-level and fall outside H_d,variant.

A.7 Determining the dominant class (paper section 2.1)

Four inputs: capture exposure (likelihood of physical loss or seizure), extractability (tamper resistance, storage encryption, key management, demonstrated firmware extraction effort), remote paths (remote intrusion and supply-chain routes), live observability (whether probing the live platform exposes the state needed to exploit it). Output: a ranked list of acquisition classes per platform with evidence and the cost ranges assumed for c_capture, c_exploit, c_public. A2-hybrid is dominant when extraction succeeds at a cost below the value of compromising the fleet; A2-pure when live observation suffices and extraction fails.

Appendix B: Test vectors

V1, toy fleet. 100 instances, A2-hybrid.

N = 1, H = 0: 1 cycle.

N = 10 (10 instances each), H = 1: 10 cycles.

N = 10, H = 0.3: three variants need independent cycles, seven are compromised from one study. Exact count = HN + 1 = 4. Continuous formula = 1 + 0.39 = 3.7. Ceiling = 4. Total cost = 4 x (c_capture + c_exploit).

V2, node ratio versus variant fraction. N = 10 variants, each with 1 shared and 9 variant-specific critical nodes. Union = 1 + 90 = 91 nodes. |S_c| = 90. H_d,node = 90/91 = 0.989 (about 0.99). If the shared node is the cheapest path for every variant (one exploit that transfers), H_d,variant = 0. The formula fed with H_d,node predicts 1 + 0.99*9 = 9.9 cycles (wrong); with H_d,variant it predicts 1 + 0*9 = 1 cycle (correct).

V3, formula versus exact count, all grid points. For N in 2..50 and integer k = HN in 0..N: exact(k) = 1 if k = 0; N if k = N; k + 1 otherwise. Assert ceil(1 + (k/N)(N-1)) == exact(k) for every case. Assert the continuous formula equals exact at k = 0 and k = N, and equals exact - H strictly between.

V4, clusters. N = 10, H = 0, non-independent variants clustered in pairs: exact count = N/2 = 5 cycles versus the formula's 1.

V5, topology. cycles = 10, k_cap = 5: captures = max(1, ceil(10/5)) = 2; total = 2 x c_capture + 10 x c_exploit. cycles = 3, k_cap = 10: captures = 1.

V6, success fraction and collapse. N = 10, f = 0.5: effective N = 5. A family designed with N = 10 but deployed as one variant: N = 1, formula degrades to the homogeneous case.

V7, A2-pure invariance. For any H in [0, 1], the A2-pure burden equals the homogeneous burden (flat).

V8, co-cheapest rule. A variant has two paths within 20% of each other, one common and one variant-specific: classified non-independent. Both variant-specific: independent. Only the single cheapest path is variant-specific and the next is more than 20% costlier: independent.

V9, largest-discount rule. A node can be reduced by 30% from studying variant A and 25% from studying variant B: the discount applied is 30%, not 55% and not multiplicative. With theta = 20%, both transfers count as material; with theta = 35%, neither does.

V10, generator check override. A family whose node-level test gives H_d,variant = 0.8 but is flagged generator-biased: reported H_d,variant = 0.

V11, endpoint H_d,node. H_d,node = 0 when every variant fails at the same nodes; 1 when every variant's critical nodes are all variant-specific and non-transferring.

Appendix C: Things to avoid

Do not describe the simulation as proof, validation, or evidence for the framework.

Do not use H_d,node in the burden formula. Use H_d,variant.

Do not cumulate transfer discounts across several studied variants.

Do not silently set any analyst-judgment default; always show and confirm it.

Do not fabricate datasets, vendors, CVEs, or citations.

Do not write working exploit code or instructions for attacking real systems.

Do not use em dashes in any prose you produce.

Do not produce a table from E9 without the MOCK DATA prefix.

Do not skip a deliverable without labeling it INCOMPLETE and explaining why.